



Pond Control Unit

An ESP32-S3 reimaging of PondPi — design & build plan

A from-scratch redesign of a backyard-pond pump and water-fill controller. The unit runs autonomous scheduling firmware on an ESP32-S3 under ESP-IDF, is configured from a separate home-server admin panel, and is built around a single integrated PCB that brings mains power, isolated logic, and three on-board triac solid-state-relay channels onto one board.

Design document · synthesized from project planning

Original concept: PondPi by Noah Robertson (MIT) · github.com/nhrobertson/PondPi

Contents

1. Overview
2. Background: the original PondPi
3. Design decisions
4. System architecture
5. Hardware requirements
6. Firmware architecture
7. Output precedence logic
8. Configuration model
9. Sync protocol & timekeeping
10. ESP-IDF project structure
11. Home server
12. PCB design
13. Indicative bill of materials
14. Plan of attack
15. Safety notice
16. Appendix: pin budget & references

1 Overview

The Pond Control Unit (PCU) controls two pond pumps and an automatic water-fill valve, with a float sensor for level detection. It replaces the Raspberry Pi Pico W at the heart of the original PondPi project with an ESP32-S3, and splits the system into three tiers: an admin browser, a home server that hosts the configuration panel, and the edge device that does the actual switching.

The defining principle of this redesign is **device autonomy**. The home server is where schedules are *edited*, but the ESP32 stores its own copy of the configuration and runs it independently. If the server reboots, the network drops, or the internet is unavailable, the pumps keep running on schedule. The server pushes configuration down; the device reports status up.

The four functional signals are unchanged from the original requirements:

- **Float sensor** — optional 3.3 V input plus an indicator LED.
- **Pump 1** — 3.3 V output driving an SSR that switches a 120 V pump, plus an LED.
- **Pump 2** — as Pump 1.
- **Water fill** — 3.3 V output driving an SSR that switches a 120 V electric valve, plus an LED.

Every output additionally has a physical switch that enables or disables that function, and every output has its own LED indicator.

2 Background: the original PondPi

PondPi is a single Raspberry Pi Pico W running MicroPython that does everything on one device. Understanding its structure clarifies exactly what is being kept, changed, and fixed.

Module	Role
<code>main.py</code>	Async entry point; initializes timers, loads the web page, connects WiFi, starts the HTTP server and the float/leak monitor tasks.
<code>hw_util.py</code>	GPIO control (outputs on pins 3/4/5, float sensor on pin 18), float-fill logic, leak detection, timer setup.
<code>timer.py</code>	A <code>Timer</code> class that checks every 60 s whether the current hour/minute/day matches and fires a callback.
<code>config_manager.py</code>	Reads/writes an INI-style <code>config.txt</code> for persistence.
<code>network_util.py</code>	WiFi connect and a hand-rolled HTTP server that parses requests and dispatches to enable/disable/set-time handlers.
<code>index.html</code>	The web UI served directly by the device.

Its float logic is worth preserving: when the float sensor stays active past a configurable *threshold* time, the water-fill output turns on; when the sensor clears, the fill stays on for an *overflow* time, then shuts off. A leak guard counts fills per day and latches everything off if more than a set number occur in 24 hours, requiring a restart.

Two gaps the remake fixes. First, the original relies on the system clock but never synchronizes time, so schedules drift — the PCU syncs via NTP and optionally a battery-backed RTC. Second, the original has no physical switches and no dedicated indicator LEDs — both are first-class requirements here.

3 Design decisions

The following choices were made up front; they shape every later section.

Decision	Choice	Rationale
Goal	Fully custom build	Maximum control and learning value; not a low-code/Home-Assistant path.
Firmware stack	ESP-IDF (C)	Native FreeRTOS, fine-grained control of tasks, NVS, drivers, and OTA.
Device autonomy	Autonomous	Device stores and runs its own schedule; survives server/network outages.
Board power	Isolated AC-DC on board	Single mains feed; fully integrated unit. Raises isolation/layout requirements.
Load switching	On-board triac SSRs (opto-triac + triac)	Compact, cheap, true solid-state switching built from discrete parts.
MCU	ESP32-S3 module	Native USB for flashing/JTAG, ample flash/PSRAM, pre-certified RF module.

4 System architecture

The system is three tiers. An admin browser talks to the home server over HTTP. The home server syncs configuration and status with the ESP32 over WiFi. The ESP32 reads 3.3 V inputs and drives 3.3 V outputs into solid-state relays that switch the 120 V loads.

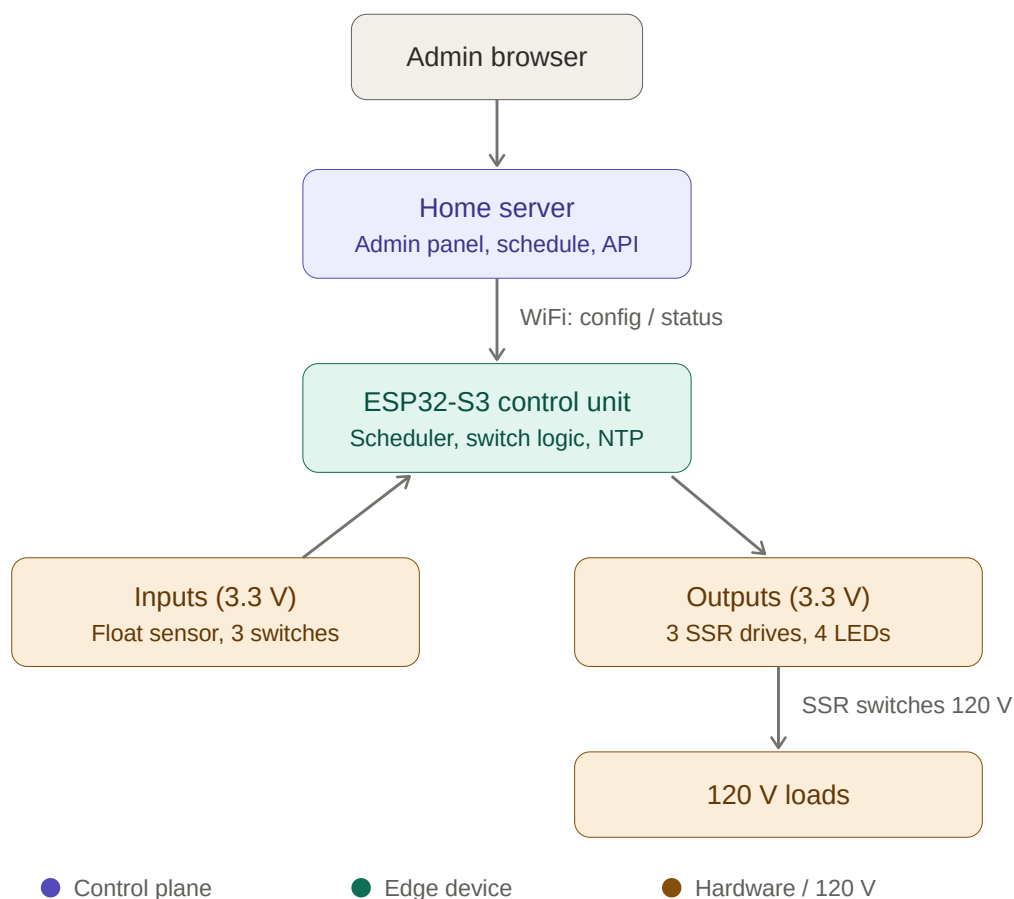


Figure 1 — Three-tier system architecture.

This separates concerns cleanly: the home server owns the human-facing editing experience, while the device owns execution. The only thing crossing the WiFi link is a small configuration blob in one direction and status in the other.

5 Hardware requirements

The I/O budget is small — roughly eleven GPIO — so any ESP32 variant suffices; the S3 is chosen for native USB and headroom. The signal set:

Signal	Direction	Notes
Float sensor	Input	Internal pull, software debounce, TVS + series resistor for the long outdoor run.
Switch 1 / 2 / 3	Input	One 3-position switch per output: <code>ON</code> / <code>OFF</code> / <code>AUTO</code> .
SSR drive 1 / 2 / 3	Output	3.3 V logic into the opto-triac LED of each channel.
LED 1 / 2 / 3	Output	Output-state indicators, driven on the logic side.
Float LED	Output	Sensor-active indicator.

Three-position switches

Rather than a plain on/off toggle, each output gets a 3-position switch. `OFF` forces the output off (a true safety lockout), `ON` forces it on (manual override), and `AUTO` hands control to the firmware schedule. The firmware reads the switch position and always honors it; the indicator LED reflects the *resolved* state so the panel never lies about what a load is actually doing. For a hardware-level guarantee, the `OFF` position can additionally interrupt the SSR control line so that a firmware fault cannot override a lockout.

6 Firmware architecture

The firmware is a set of cooperating FreeRTOS tasks. Because the device is autonomous, the schedule lives in non-volatile storage (NVS) and runs with or without the network.

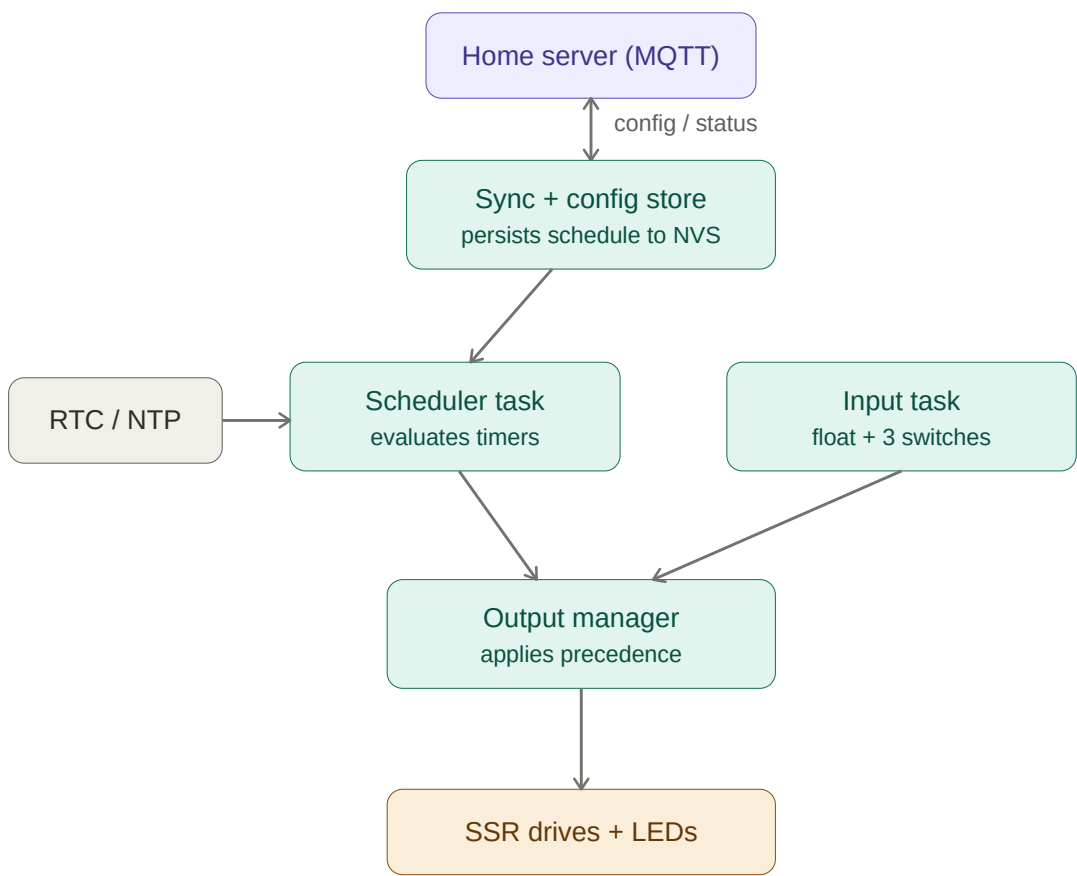


Figure 2 — FreeRTOS task and data flow inside the autonomous firmware.

Task	Responsibility
Sync + config store	Receives configuration over MQTT, validates it, and persists it to NVS. Notifies the scheduler to reload.
Scheduler	Wakes roughly once a minute, compares each output's on/off timer and active days against the RTC, and sets the desired AUTO state.
Input task	Debounces the float sensor and reads the three switch positions; raises events for the output manager.
Output manager	Resolves every output through the precedence rules below and drives the SSR control lines and indicator LEDs. Also owns the float-fill and leak logic.

Shared state (the live configuration and resolved output states) is guarded by a mutex; tasks communicate desired changes through queues or event groups so no task touches a GPIO that another task owns.

7 Output precedence logic

The output manager is where the physical switch, the schedule, the float logic, and leak protection all collide. Defining the priority order is the single most important correctness decision in the firmware. For each output, resolve from highest priority to lowest:

1. **Leak lockout active** → **force OFF**. Latched; clears only on an explicit reset, exactly like the original's leak guard.
2. **Physical switch in OFF** → **force OFF**. The hardware safety lockout.
3. **Physical switch in ON** → **force ON**. Manual override.
4. **Physical switch in AUTO** → **defer to firmware**. Pumps follow their schedule; the fill valve follows the float-fill logic plus its own schedule.

The indicator LED reflects the *resolved* state, never the raw command. A pump locked off by its switch shows OFF even if the schedule wants it on — the panel stays honest about what each load is actually doing.

8 Configuration model

The device owns one JSON configuration blob stored in NVS — the JSON-native successor to PondPi's `config.txt`. The server publishes it; the device persists it. A monotonically increasing `version` counter lets the device tell whether an incoming message is newer than what it already holds. ESP-IDF's bundled `cJSON` parses it.

```
{
  "version": 4,
  "outputs": [
    { "id": 1, "on": "14:04", "off": "14:06", "days": ["Mon","Tue","Wed","Thu","Fri"] },
    { "id": 2, "on": "06:00", "off": "20:00", "days": ["Sat","Sun"] },
    { "id": 3, "on": null, "off": null, "days": [] }
  ],
  "float": { "threshold_min": 10, "overflow_min": 30, "max_fills_per_day": 3 }
}
```

Times are stored in 24-hour form to avoid the AM/PM parsing fragility of the original. Output 3 (water fill) may carry a null schedule when it is driven purely by the float sensor.

9 Sync protocol & timekeeping

MQTT topics

MQTT with retained messages is a natural fit for an autonomous device: a freshly rebooted unit receives the latest schedule the instant it subscribes, with no live server interaction required. A Last-Will message gives the server a reliable online/offline signal.

Topic	Publisher	Notes
pond/config	server	Retained JSON blob; device writes to NVS when version increases.
pond/status	device	Retained; resolved output states and last-synced version.
pond/availability	device	MQTT Last-Will: online / offline .
pond/cmd	server	Optional momentary commands, e.g. clear leak lockout.

Timekeeping

On boot the device synchronizes time via SNTP and sets its timezone (for example EST5EDT) so that local schedule comparisons are correct across daylight-saving changes. A DS3231 RTC with a coin-cell backup is optional insurance: it keeps accurate time through internet outages and across power loss, addressing the clock-drift gap in the original design.

10 ESP-IDF project structure

One component per concern keeps the build clean and maps directly onto the task diagram.

```
pond-control-unit/  
├─ main/                app_main: nvs + wifi + sntp init, task launch  
├─ components/  
│   ├─ config/          NVS read/write, cJSON schema parse/validate  
│   ├─ scheduler/       per-minute timer evaluation, Timer model  
│   ├─ input/           GPIO debounce: float + 3-position switches  
│   ├─ output_manager/  precedence resolver, drives SSR + LED GPIO  
│   ├─ timekeeping/     SNTP + timezone, optional DS3231 over I2C  
│   └─ net/             wifi provisioning + esp-mqtt sync  
└─ CMakeLists.txt
```

Phases 0–4 of the plan (Section 14) produce a fully working autonomous controller on the bench using only the `config`, `scheduler`, `input`, `output_manager`, and `timekeeping` components — the network layer is added last.

11 Home server

The home server's job is deliberately small: edit schedules and publish the resulting configuration. It is not in the real-time control path. A practical stack is an MQTT broker (Mosquitto) plus a lightweight web application (FastAPI or Node) that:

- renders an admin panel for editing each output's on/off times and active days, and the float threshold/overflow/leak parameters;
- writes a new retained `pond/config` message with an incremented `version` whenever the user saves;
- subscribes to `pond/status` and `pond/availability` to show live device state and online status.

Because the configuration is retained on the broker, the server itself can be down whenever it is not actively being used — the device already holds everything it needs.

12 PCB design

The board is designed as two electrically separated domains on one piece of FR4, because it mixes 3.3 V logic with switched 120 V. A logic (SELV) domain and a mains domain are split by an isolation barrier; the only things that cross it are the isolated AC-DC converter and the opto-triacs.

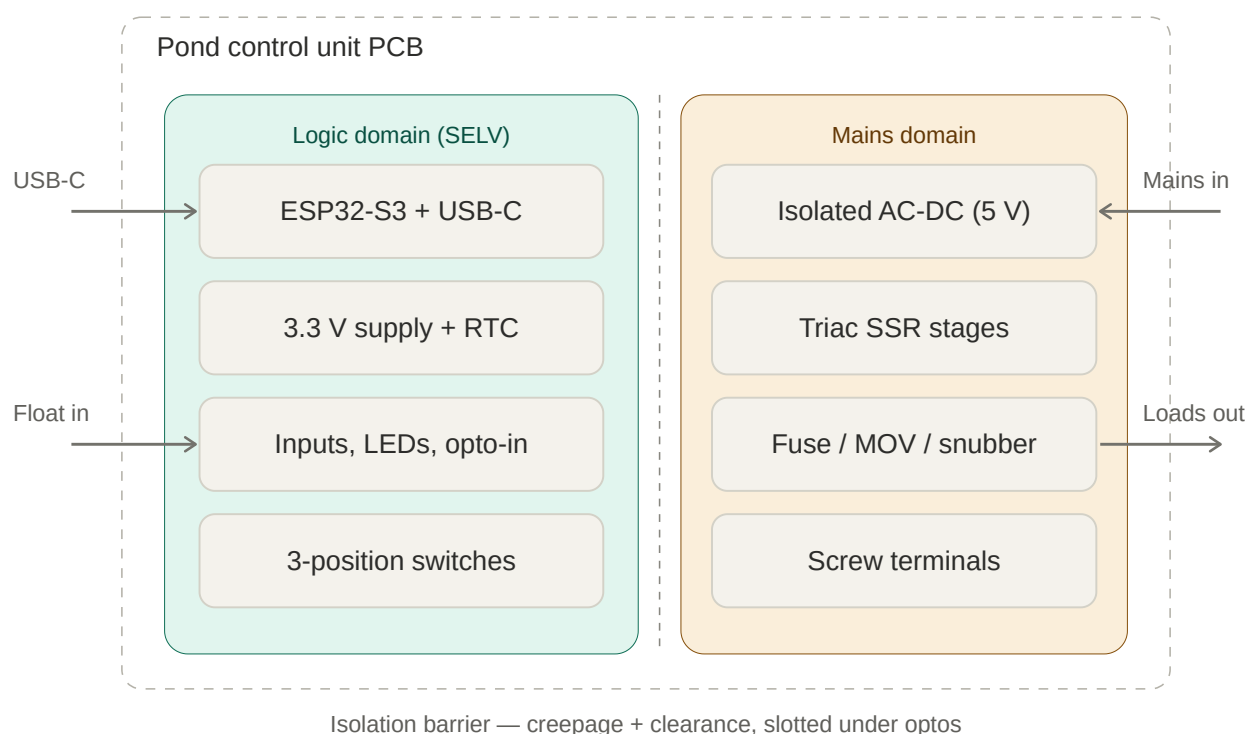


Figure 3 — PCB partitioned into a logic domain and a mains domain.

Power chain

Mains enters through an L/N/PE screw terminal behind a slow-blow fuse, with an MOV across line-to-neutral. It feeds an isolated AC-DC module sized with headroom — the ESP32-S3's WiFi current spikes to roughly half an amp, and the opto and indicator LEDs add to that, so a 5 W (1 A) module such as the HLK-5M05 is a safer pick than the 3 W parts that run right at their limit. From the isolated 5 V rail, a small buck converter steps down to 3.3 V (more efficient and cooler than an LDO), with bulk capacitance on both rails and tight decoupling at the module. With a protective-earth conductor present, the board is treated as earthed Class I and PE is bonded to the enclosure.

Per-channel triac SSR stage (×3)

Each of the three channels is identical. A GPIO drives the opto-triac's LED through a series resistor; the opto's output triac fires the gate of a power triac, which sits in series with the line feeding the load. The optocoupler is what makes a single-board design safe — the isolation barrier runs straight through it.

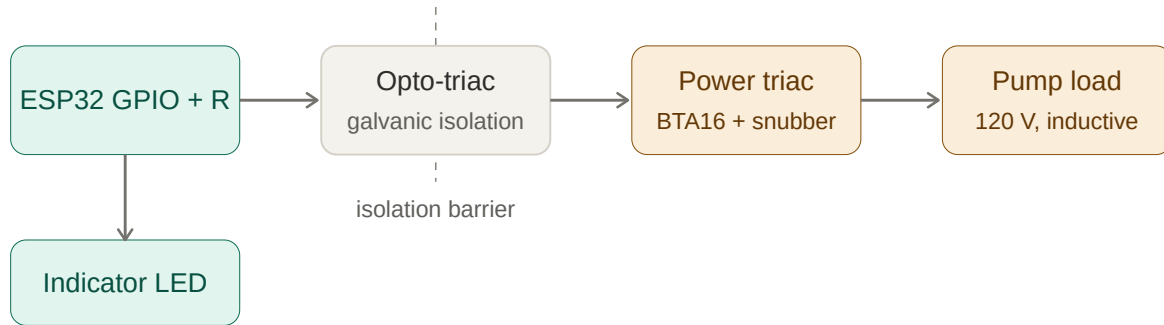


Figure 4 — One of three identical triac SSR channels.

A BTA-series triac is the right family because the "A" denotes an isolated tab, which makes heatsinking simple. Fountain pumps are typically under an amp, so a BTA16 leaves large margin and runs cool. One nuance from the inductive-load choice: zero-cross opto-triacs are tuned for resistive loads, so for motor loads either use a snubberless triac (the BTA "W" suffix) with an RC snubber, or a random-phase opto-triac, and size the snubber against the triac's commutating dv/dt rating. An RC snubber — on the order of $100\ \Omega$ in series with a 100 nF X-rated capacitor — across each triac is not optional with inductive loads. The indicator LED stays on the logic side, driven straight from the GPIO, so it shows the resolved state without touching mains.

Isolation & safety

No copper trace should bridge the two domains. The isolated AC-DC transformer and the opto-triacs provide the only galvanic crossings. Across that barrier the board needs reinforced-insulation spacing: a common rule of thumb for a 120 V line is on the order of 6–8 mm of creepage, helped by milling a slot through the board under the optos and the AC-DC module so surface tracking cannot shortcut it.

Verify spacings against the actual standard. Exact creepage and clearance values depend on the safety standard you design to, the pollution degree, and the board material group. Treat the figures above as a starting target only, and confirm them against the relevant tables for your region before committing the layout.

PCB process

- **Tooling:** KiCad for schematic capture and layout. Pull libraries for the WROOM module, the AC-DC module, the opto/triac parts, the DS3231, and rated terminal blocks.
- **Layers:** a 2-layer board is feasible with care, but 4 layers make the power/ground planes and RF routing much easier and are worthwhile for a board carrying mains.
- **Layout:** partition the domains, route the isolation slot, widen the mains traces for pump current (2 oz copper or wide pours), and keep copper out from under the module's antenna at the board edge. Add test points on every rail.

- **Fabrication:** JLCPCB or PCBWay, with optional assembly of the SMT parts. Bring rev A up in stages — populate and verify the isolated 5 V and 3.3 V rails *before* the ESP32 is ever populated.

13 Indicative bill of materials

Candidate parts by functional block. Values and exact part numbers should be confirmed against datasheets and the final load ratings.

Block	Candidate part	Notes
MCU module	ESP32-S3-WROOM-1 (N8 / N8R8)	Native USB; antenna keep-out at board edge.
USB	USB-C receptacle + USBLC6-2 ESD	Wired to the S3 native USB for flash/JTAG.
AC-DC	HLK-5M05 (5 V / 5 W isolated)	Headroom over WiFi current peaks.
3.3 V regulator	Buck (e.g. TPS562201 / MP1584)	LDO (AMS1117-3.3) acceptable but warmer.
RTC	DS3231SN + CR2032 holder	Battery-backed time over I2C.
Opto-triac driver ×3	MOC3052 (random-phase) or MOC3062 (zero-cross)	Choice depends on load and snubber design.
Power triac ×3	BTA16-600BW (snubberless, isolated tab)	Large margin over typical pump current.
Snubber ×3	≈100 Ω 0.5 W + 100 nF X2 cap	Required for inductive motor loads.
Input protection	Fuse + holder, MOV rated for the 120 V line	Slow-blow fuse sized for board + inrush.
Float input	TVS (e.g. SMAJ5.0A) + series resistor + RC	Protects the long outdoor sensor run.
Indicators	4 × LED + current-limit resistors	Three outputs plus float, on the logic side.
Switches ×3	SPDT center-off (ON/OFF/AUTO)	Panel- or board-mounted to the faceplate.
Terminals	Mains-rated screw terminals (300 V)	L/N/PE in, three load outputs, float in.

14 Plan of attack

Firmware is validated on a dev board first; the PCB is designed only once the circuit is proven on prototype hardware. Phases 0–4 yield a working autonomous controller on the bench with no server and no custom board.

Phase	Work
0 — Toolchain	Install ESP-IDF, <code>idf.py set-target esp32s3</code> , flash the blink example to confirm the board and USB-JTAG.
1 — Bare I/O	Drive one LED; read a 3-position switch and the float sensor with software debounce; log states. No logic yet.
2 — Timekeeping	SNTP sync, set timezone, confirm local time across a reboot; add the DS3231 if offline accuracy is wanted.
3 — Scheduler + output manager	Port the timer model, load config from NVS (hardcoded blob first), implement the precedence resolver; run fully offline.
4 — Float-fill + leak	Port the float-fill threshold/overflow logic and the daily fill-count leak guard.
5 — Connectivity	WiFi + esp-mqtt; subscribe to retained <code>pond/config</code> , persist on version bump, publish status and Last-Will.
6 — Prototype circuit	Build the complete circuit on protoboard — every input, real switches and LEDs, real SSR modules — and freeze the schematic.
7 — Schematic capture	Enter the schematic in KiCad and run ERC.
8 — PCB layout	Partition the domains, route the isolation slot, set creepage/clearance, widen mains traces, honor the antenna keep-out; run DRC.
9 — Fab + bring-up	Fabricate and assemble rev A; bring up power rails first, then the rest.
10 — Hardware firmware	Bring-up firmware; spin rev B for fixes and to fold in the on-board AC-DC if deferred.
11 — Home server	Mosquitto plus the admin panel that edits and publishes the retained config.
12 — Enclosure	IP-rated box, sealed switch/LED faceplate, GFCI on the pond circuit, conformal coating, soak test.

15 Safety notice

This project switches mains voltage near water. Both are independently capable of causing serious injury or death, and the combination demands real caution.

- Have the mains section of the design reviewed by someone qualified in mains PCB work before fabrication.
- Bring rev A up through an isolation transformer and, ideally, a current-limited supply; never make contact with the board while it is powered from mains.
- Confirm all creepage and clearance distances against the safety standard applicable in your region.
- Install the unit behind a GFCI / RCD, bond protective earth to the enclosure, and use an IP-rated, conformally-coated assembly for outdoor use.
- Follow your local electrical code; in many jurisdictions permanent mains wiring must be performed or inspected by a licensed electrician.

This document is a design plan, not a certified or inspected design. You are responsible for the safety of anything you build from it.

16 Appendix

Pin budget

Category	Count	Lines
Inputs	4	Float sensor + three switch positions (or 6 lines if each 3-position switch uses two GPIO).
SSR drives	3	Opto-triac LED per channel.
Indicator LEDs	4	Three outputs plus float.
I2C (RTC)	2	SDA / SCL to the DS3231.

Roughly 13–16 GPIO including the RTC bus — comfortably within the ESP32-S3's available pins.

References

- Original project: PondPi by Noah Robertson — github.com/nhrobertson/PondPi (MIT License).
- ESP-IDF programming guide and component reference (Espressif).
- Opto-triac driver application notes (e.g. ON Semiconductor MOC30xx series) for inductive-load snubber design.